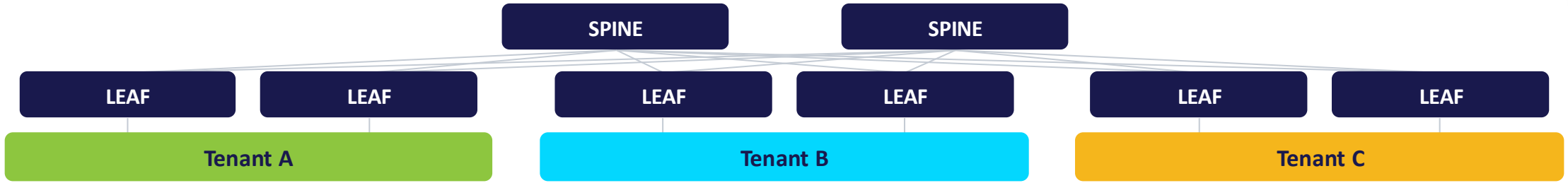
Secure Multi-Tenancy for Open AI Clusters

Manish Vachharajani, CTO, Hedgehog

Art Fewell, Technical Marketing Engineer, Hedgehog

OCP Global Summit Breakout Session



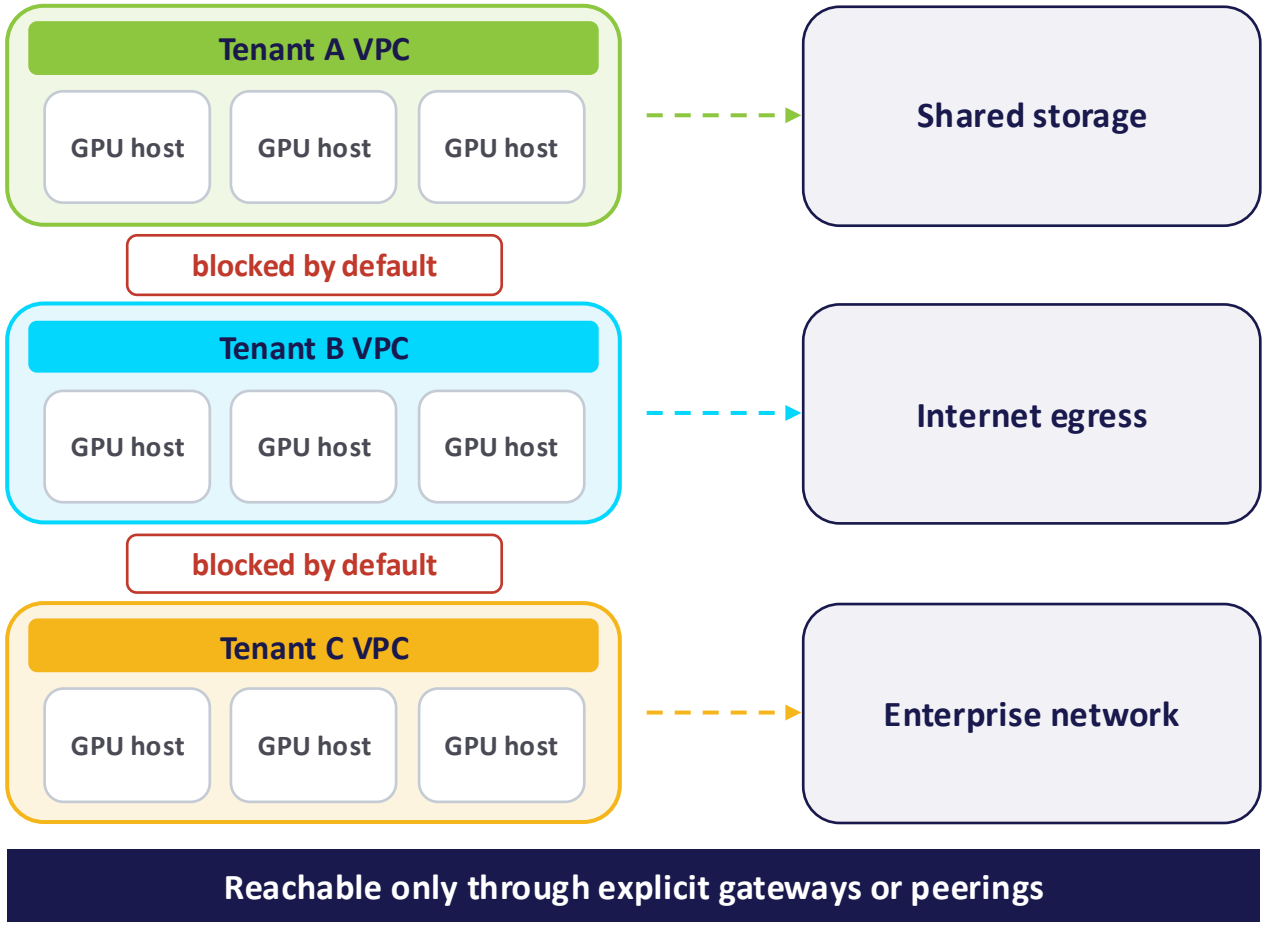
Cloud Set The Expectation

Users expect cloud-like tenancy

A tenant should get:

- An isolated private network
- Self-service provisioning
- Explicit peering to shared services
- Controlled internet or enterprise access
- Rapid teardown and reprovisioning
- Auditability of every change

The mental model is VPC, not VLAN ticket.



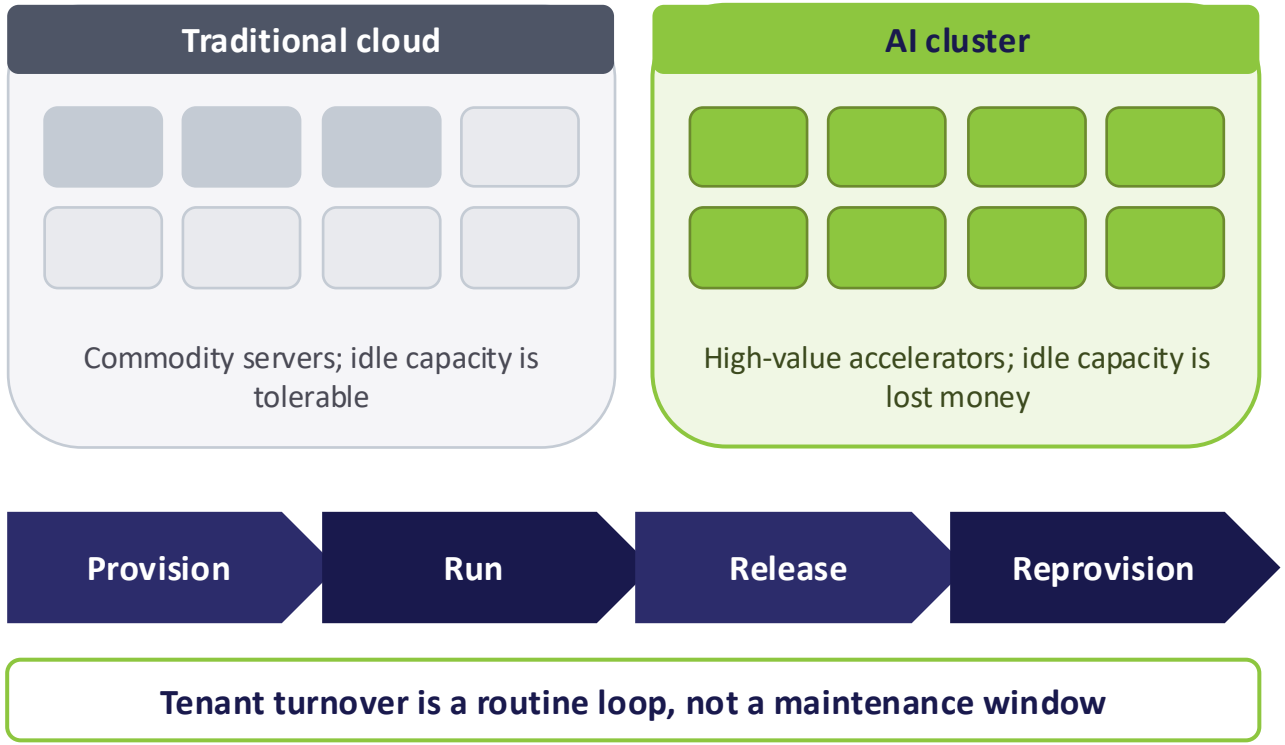
AI Changes The Economics

GPU clusters cannot be treated like ordinary shared compute

AI infrastructure is:

- Expensive
- Power-constrained
- Performance-sensitive
- Rapidly depreciating
- Often subject to privacy or sovereignty constraints
- Expected to stay highly utilized

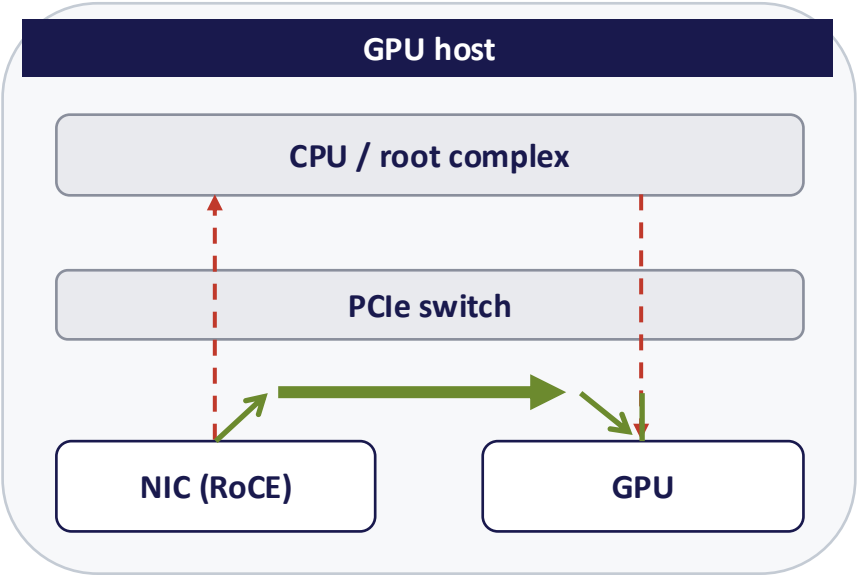
Result: tenants must be provisioned, isolated, changed, and removed quickly.



Why Host-Based Isolation Breaks

Traditional isolation lives on the host. AI wants the host out of the way.

Approach	Why it works in cloud	Why it fails for AI
VMs	Strong isolation	IOMMU / ACS can block GPU-direct performance
Containers	Lightweight tenancy	Weaker boundary; shared kernel and cluster-wide resources
Host overlays	Flexible networking	Adds host dependency to high-performance paths
DPUs	Offload isolation	Practical on front-end; difficult at backend GPU scale



Red: through CPU root complex. Green: direct PCIe peer-to-peer.

Key point: RoCE backend performance limits where isolation can live.

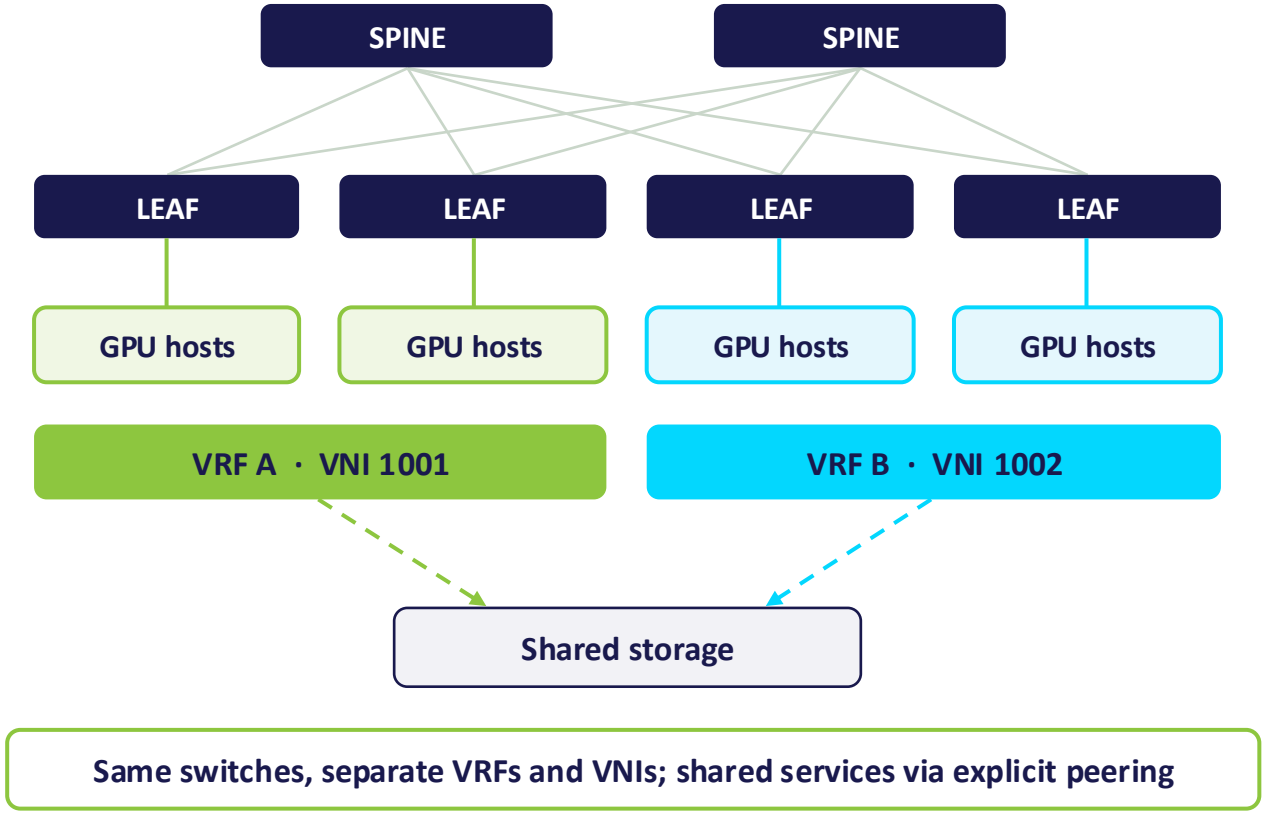
The Isolation Boundary Moves To The Fabric

The physical network can enforce tenant boundaries

Using open Ethernet fabric capabilities:

- BGP/EVPN control plane
- VXLAN data plane
- Per-tenant VRFs
- VNI allocation
- Route policy
- ACLs and anti-transit controls
- Controlled external connectivity

Tenant isolation becomes a fabric service.



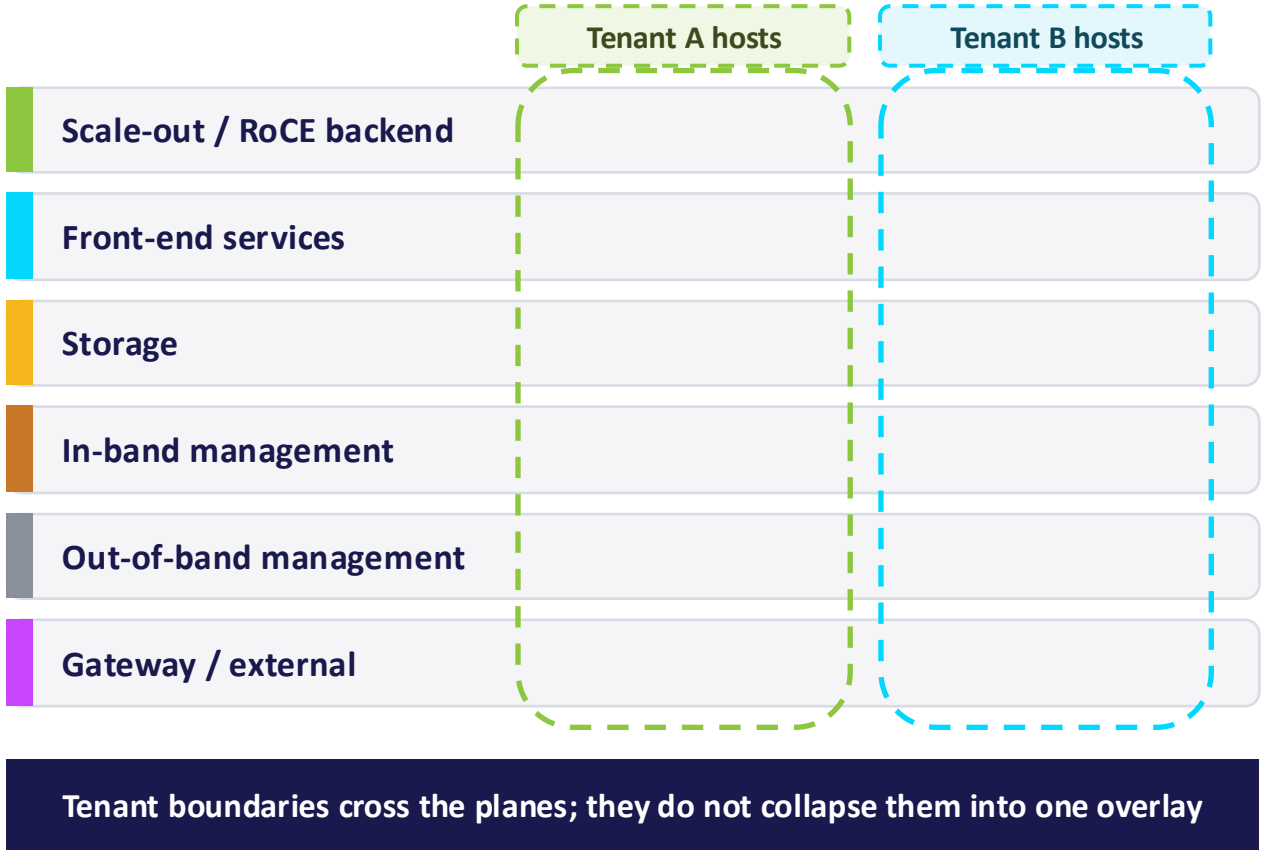
OCP-Aligned AI Fabric Context

Open AI clusters already separate traffic by role

Reference architecture fabric domains include:

- Scale-out / RoCE backend
- Front-end services
- Storage
- In-band management
- Out-of-band management
- Gateway and external connectivity tiers

Secure tenancy must preserve both isolation and performance.



VPC Semantics On Physical Switches

Expose VPC intent; hide switch mechanics

Operator intent

- Create tenant VPC
- Attach hosts
- Peer to shared storage
- Permit selected external routes
- Add NAT or gateway access
- Remove tenant cleanly

Deterministic controller

validate → reconcile

Fabric implementation

- VRFs
- VNIs
- Route targets
- Route maps
- ACLs
- BGP/EVPN state

Thousands of lines of generated switch state — reviewed by software, not by hand

The operator should not hand-edit the implementation layer.

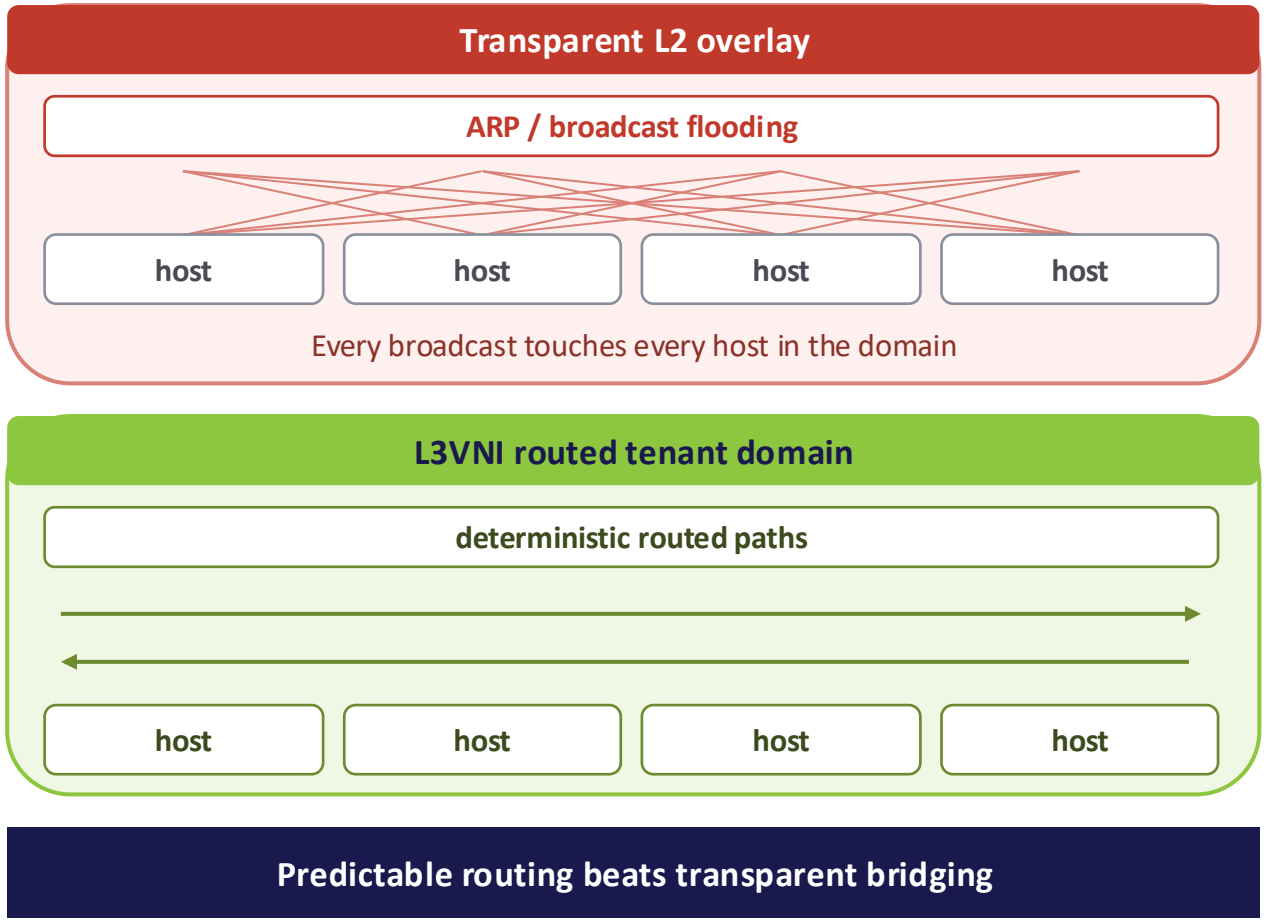
L3VNI Is A Feature, Not A Compromise

AI fabrics should avoid unnecessary L2 behavior

Why L3VNI fits AI clusters:

- Per-tenant controlled routing
- Fabric IP enforcement
- No large L2 broadcast domain
- Less ARP / broadcast pressure on RoCE paths
- Better alignment with high-density switch silicon
- Clearer policy and anti-transit enforcement

For RoCE-oriented fabrics, predictable routed behavior beats transparent L2.



Peering Without Transit Leaks

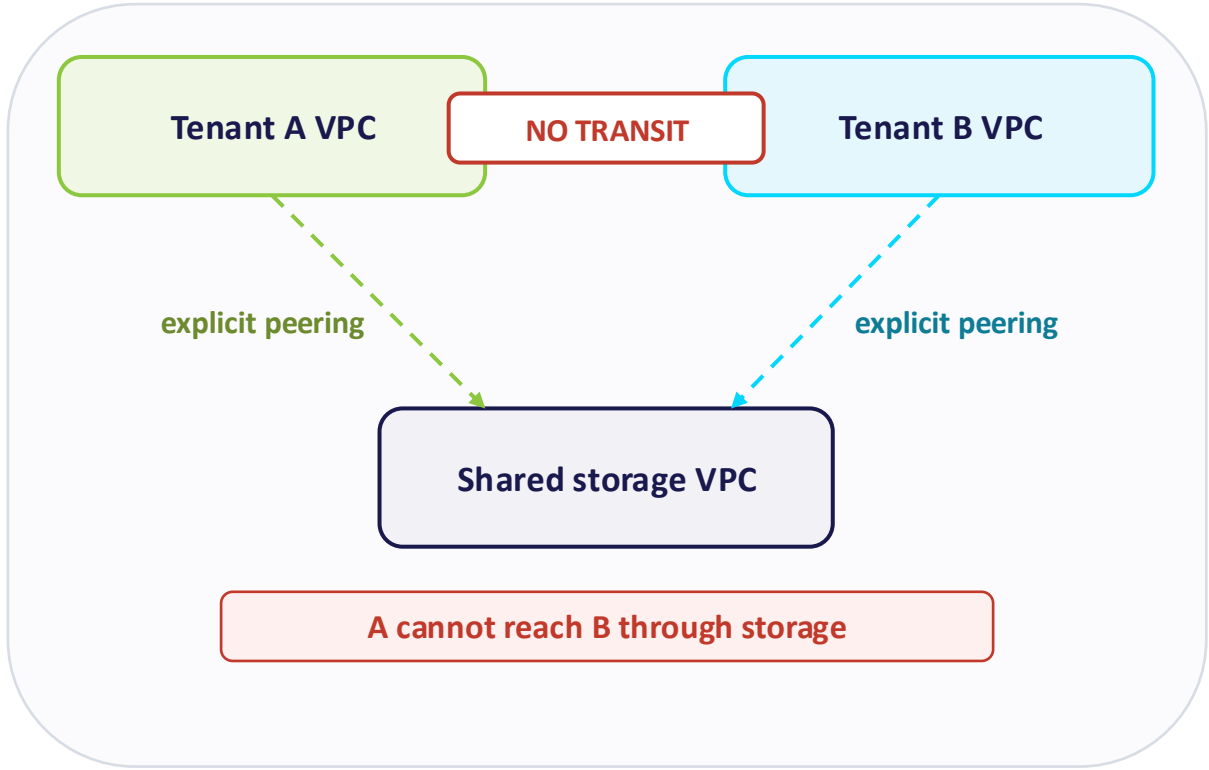
Peering must be explicit, bounded, and non-transitive

Common peerings

- Tenant → shared storage
- Tenant → model repository
- Tenant → observability
- Tenant → enterprise services
- Tenant → internet egress

Required controls

- Route leaking only where intended
- ACLs for permitted prefixes
- Anti-transit policy
- Default deny between tenants
- Auditable connectivity state



A shared service must not become a bridge between tenants.

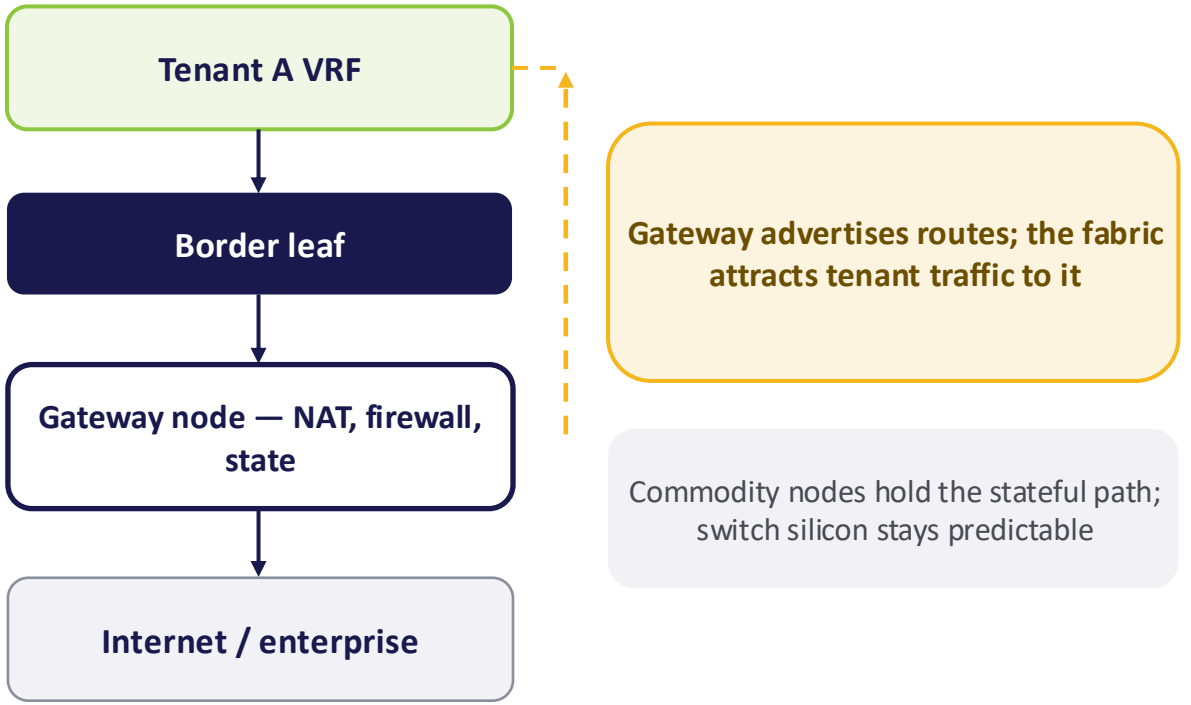
Gateway Tier For Stateful Services

Switches forward. Gateways provide state.

Gateway services can provide:

- Stateful NAT / masquerade
- 1:1 NAT for overlapping tenant IPs
- Firewall policy
- Internet egress
- Enterprise or partner connectivity
- Controlled route advertisement into tenant VRFs

Reached through normal routing, not special-case fabric hacks.



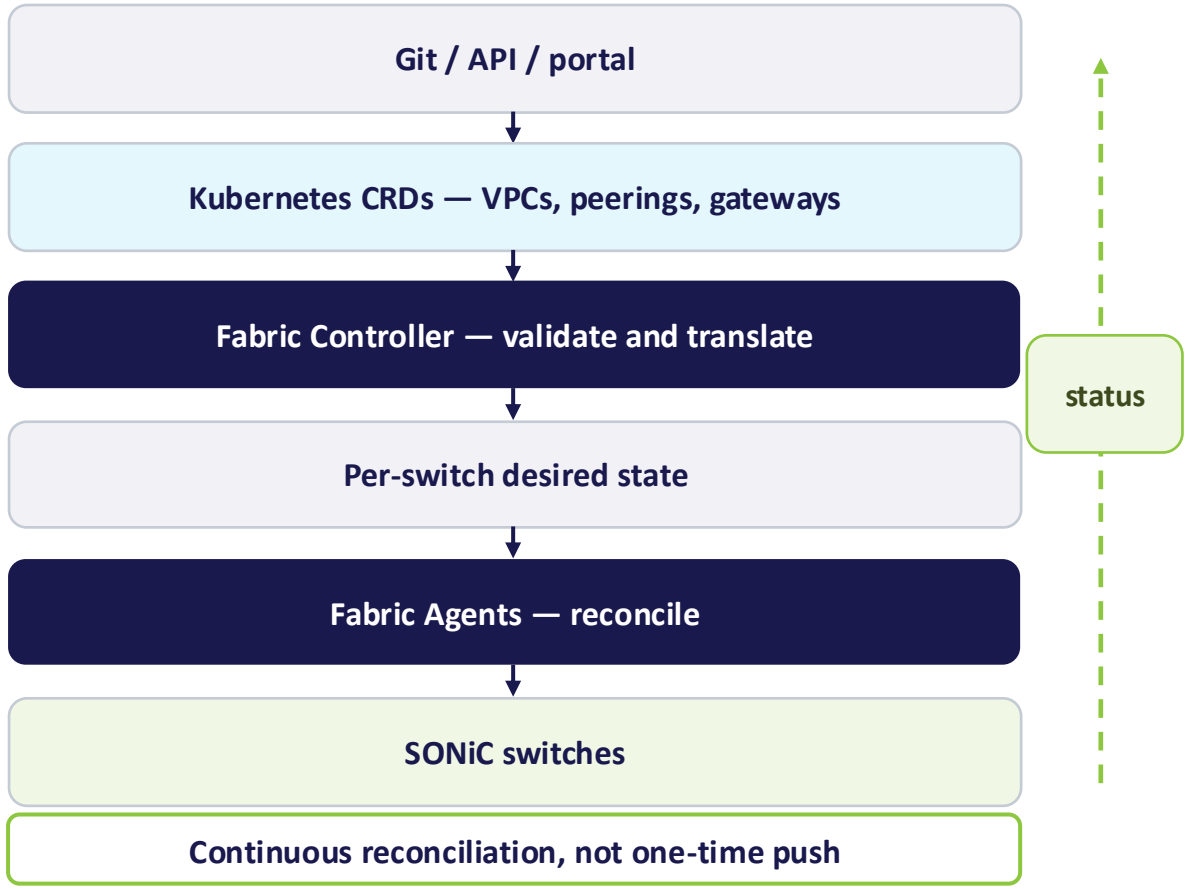
Stateful services do not belong in switch silicon

Declarative Control Plane

Secure multitenancy requires deterministic automation

- Kubernetes API as the fabric interface
- CRDs for VPCs, peerings, gateways, switches, wiring
- Fabric Controller translates intent
- Fabric Agents reconcile switch state
- Validation runs before changes
- Drift is continuously corrected

Same intent, same output, every time.



The Right Layer For AI Ops And GitOps

AI should generate intent, not switch configuration

Bad pattern

- Natural language request
- ≈1,000 lines of generated switch configuration
- Too complex to review
- No correctness guarantee

Good pattern

1. Natural language → small VPC manifest
2. Human reviews intent
3. Git records approval
4. Controller enforces configuration
5. Reconciliation maintains state

AI generates intent. Humans approve intent. Deterministic software enforces intent.

Example Tenant Lifecycle

Tenant lifecycle as auditable fabric intent

Provision

- Create VPC
- Attach hosts
- Peer to storage
- Add gateway access

Operate

- Validate reachability
- Monitor drift
- Enforce policy

Remove

- Delete peerings
- Release hosts
- Remove routes and NAT state
- Preserve audit trail

reprovision — the loop closes and repeats

Git commit | **Validation** | **Reconciliation** | **Audit log**

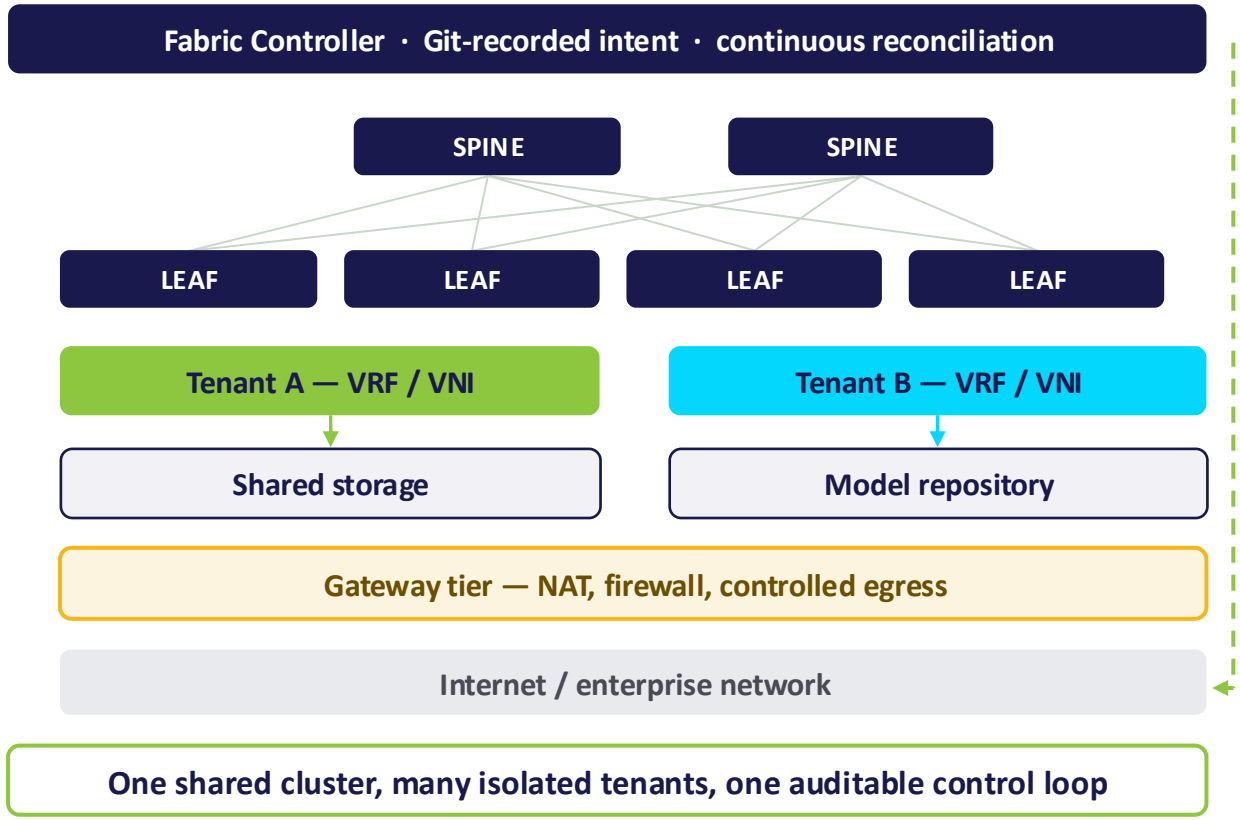
Reprovisioning becomes a controlled workflow, not a maintenance window.

Closing Takeaways

Secure multi-tenancy for AI clusters requires a different boundary

- Cloud VPCs define the user expectation
- AI performance weakens host-based isolation models
- The physical fabric can enforce tenant isolation
- VPC-style intent maps cleanly to VRF, EVPN/VXLAN, policy, and gateways
- Declarative automation makes the model operationally safe
- OCP-aligned Ethernet fabrics can support open, auditable multitenant AI infrastructure

Do not make switch configuration easier to generate; make it unnecessary to see.




Thank
You!
OCP
GLOBAL
SUMMIT
October 12-15, 2026
San Jose, California